

Complexidade Ciclomática

Prof. Thiago Jabur Bittar

O que é Complexidade Ciclomática?

- Também chamada de Complexidade Condicional (CC)
- Métrica de qualidade de código criada por Thomas McCabe (1976)
- Mede a complexidade lógica de uma parte do software, como uma classe ou método
- Baseia-se na quantidade de caminhos independentes de execução

Ideia central

Mais decisões → mais caminhos possíveis → maior complexidade

if

+1

for

+1

while

+1

switch / case

+1

A CC não diz se o código é “bom” ou “ruim” sozinha.

Ela mede **uma dimensão** da complexidade.

Caminhos independentes

- Um caminho independente introduz pelo menos uma nova condição ou um novo conjunto de comandos no fluxo
- A complexidade ciclomática representa o número de caminhos independentes no grafo de fluxo
- Por isso, decisões no código são o principal fator de aumento da CC.

Como calcular

Forma prática

$CC = \text{número de decisões} + 1$

Pelo grafo

$$M = E - N + 2P$$

- M = complexidade ciclomática / caminhos independentes
- E = número de arestas
- N = número de nós (vértices)
- P = número de componentes conexas; em uma função isolada, $P = 1$

Exemplo em Java

```
public boolean method() {  
    if (x > 4) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

Cálculo

1 decisão + 1

CC = 2

Pelo grafo

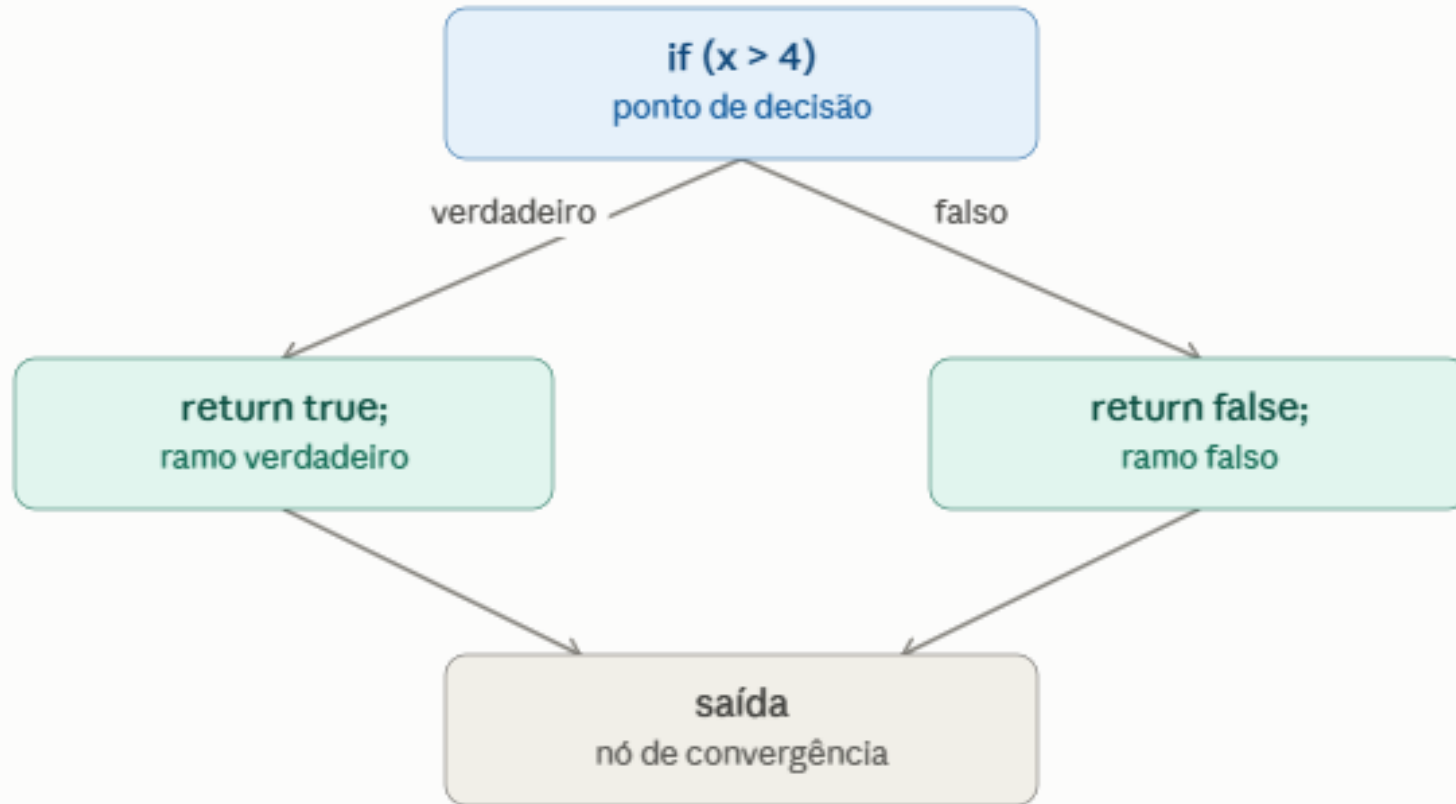
$E = 4$

$N = 4$

$P = 1$

$CC = 4 - 4 + 2(1) = 2$

Grafo de fluxo: enxergando a CC



- Uma decisão cria dois caminhos de saída.
- Para um único `if`: $CC = 1 + 1 = 2$.

Exemplo progressivo

1 decisão

```
if (x > 4) ...  
CC = 1 + 1 = 2
```

2 decisões

```
if (x > 4) ...  
if (y > 2) ...  
CC = 2 + 1 = 3
```

3 decisões

```
if ...  
while ...  
if ...  
CC = 3 + 1 = 4
```

O aninhamento pode tornar o código difícil de ler,
mas a CC é determinada pela quantidade de decisões contabilizadas.

O que conta como decisão?

- ``if`` / ``else if``
- ``for`` e ``while``
- ``switch`` / ``case`` (conforme a convenção adotada)
- Operadores lógicos como ``&&`` e ``||`` podem ser tratados de modo diferente por definições e ferramentas.

Atenção

Ao comparar valores de CC, verifique a convenção usada pela ferramenta.

Complexidade Ciclomática e testes

- A CC indica quantos caminhos independentes devem ser considerados para a **cobertura de caminhos independentes**
- Funções com muitos desvios tendem a exigir mais cenários para serem testadas
- Cada caminho independente deve ser analisado para definir os casos de teste adequados
- CC não determina sozinha a quantidade final de testes: um teste pode exercitar mais de um caminho

CC e qualidade de código

Leitura

Mais decisões podem dificultar a compreensão do código.

Manutenção

Código muito ramificado pode ser mais difícil de alterar.

Bugs

Mais caminhos significam mais cenários a considerar.

Testes

Maior CC aumenta a quantidade de caminhos independentes a analisar.

CC é uma métrica — não um diagnóstico completo da qualidade do software.

Valores de referência

Use como heurística, não como regra absoluta

1–5

Baixa
complexidade

6–10

Complexidade
moderada

> 10

Atenção /
investigar

> 20

Alta complexidade

- Os limites variam conforme a organização, linguagem, ferramenta e contexto.
- Uma função com CC alta merece análise, mas o número isolado não determina se o código é adequado.

Como reduzir a complexidade

Métodos menores

Dividir responsabilidades em funções menores e coesas.

Refatoração

Simplificar condições e retirar lógica desnecessariamente aninhada.

Polimorfismo

Em alguns casos, substituir grandes cadeias condicionais por abstrações e comportamentos especializados.

Ferramentas

SonarQube / SonarCloud

Análise estática e acompanhamento de métricas e problemas de código.



PMD / Checkstyle

Ferramentas tradicionais de análise estática para Java, com regras configuráveis.



Lizard

Análise de complexidade e métricas em várias linguagens. **Open source.**



Atualmente modelos de IA também calculam CC de códigos.

Lizard

- 
- <https://github.com/terryyin/lizard>
 - Projeto open source de análise de CC de várias linguagens
 - 2.500 estrelas no GitHub

Conclusão

- Complexidade Ciclométrica mede a complexidade lógica do fluxo de execução
- Mais decisões → mais caminhos independentes
- CC ajuda a identificar métodos que merecem revisão, refatoração e atenção nos testes
- Use a CC junto com outras métricas e com análise humana do código